

# Fundamentos del Testing de Software Full-Stack y Documentación

Este documento resume los tipos esenciales de pruebas y la herramienta **Swagger** que todo desarrollador (enfocado en React y Express.js) debe aplicar para garantizar la calidad y robustez del código y su documentación.

## I. La Pirámide de Pruebas: Estructura y Prioridad

La pirámide de pruebas es una guía visual que indica cómo se deben distribuir los esfuerzos de *testing* para lograr un equilibrio entre **rapidez**, **aislamiento** y **cobertura**.

| Nivel de la Pirámide             | Velocidad | Aislamiento | Énfasis                        |
|----------------------------------|-----------|-------------|--------------------------------|
| Cima (E2E / Manual)              | Lento     | Bajo        | Flujo completo del usuario.    |
| Medio (Integración / Componente) | Medio     | Medio       | Interacción entre módulos.     |
| Base (Unitario)                  | Rápido    | Alto        | Lógica individual y funciones. |

### TDD: Desarrollo Guiado por Pruebas

El **TDD** (*Test-Driven Development*) es una práctica donde **primero se escriben los tests** (que fallan), y luego se escribe el código funcional necesario para que esos tests pasen. Esta metodología se alinea con la filosofía de priorizar las Pruebas Unitarias.

## II. Tipos de Pruebas Clave para el Desarrollador

El alcance de las pruebas se clasifica en los siguientes tipos:

### 1. Unit Test (Prueba Unitaria)

- Objetivo:** Probar la **unidad mínima** de código (función, método) de manera completamente **aislada**.
- Aislamiento:** Se utiliza el *mocking* para simular dependencias externas (DB, APIs, services).

- **Aplicación Full-Stack:**
  - **Backend (Express):** Funciones de utilidades, lógica de negocio pura del *Service* o validaciones.
  - **Frontend (React):** *Custom Hooks* o funciones de *helpers* que no dependen de la interfaz de usuario.

## 2. Component Test (Prueba de Componente)

- **Objetivo:** Probar un componente de React en **aislamiento**, verificando que se renderice correctamente en diferentes estados (*props*).
- **Aplicación (React):** Asegurar que un componente específico (ej. un modal o una tabla) muestre o esconda elementos basándose en sus entradas (*props*).

## 3. Integration Test (Prueba de Integración)

- **Objetivo:** Verificar que las **interfaces** y diferentes partes del sistema se relacionan correctamente.
- **Aplicación Full-Stack:**
  - **Backend (Express):** Pruebas de **rutas de la API** que simulan solicitudes HTTP (ej. con **Supertest**) para validar la cadena de **Middleware** → **Controller** → **Service**.
  - **Frontend (React):** Probar la interacción de un componente con una fuente de datos *mockeada* o la comunicación entre un componente padre e hijo.

## 4. End-to-End (E2E) Test

- **Objetivo:** Simular un **flujo completo del usuario de principio a fin** (Prueba de **caja negra**). Valida el comportamiento de toda la aplicación.
- **Herramientas Comunes:** Cypress, Playwright.
- **Aplicación Full-Stack:** Flujos críticos como el registro de un usuario, el proceso de *checkout* o el inicio de sesión.

---

## III. SWAGGER / OpenAPI: Documentación y Contrato de API

**Swagger** se refiere al conjunto de herramientas basadas en la **Especificación OpenAPI (OAS)**, crucial para la documentación de APIs REST.

| Concepto | Función | Importancia para el Testing |
|----------|---------|-----------------------------|
|          |         |                             |

|                                     |                                                                                                                                                          |                                                                                                                                                                |
|-------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Especificación OpenAPI (OAS)</b> | Es el <b>archivo de contrato</b> (YAML/JSON) que describe formalmente toda la API (rutas, parámetros de entrada, <i>schemas</i> y códigos de respuesta). | Define el contrato que las <b>Pruebas de Integración</b> deben validar y que el <i>Frontend</i> debe cumplir.                                                  |
| <b>Swagger UI</b>                   | Es la interfaz gráfica interactiva generada a partir del archivo OAS.                                                                                    | Permite a los desarrolladores y <i>testers</i> probar y consumir los <i>endpoints</i> de la API en vivo sin necesidad de herramientas externas (como Postman). |

## IV. Prompt Engineering Avanzado para Testing

Para generar pruebas de alta calidad con IA, debes aplicar técnicas avanzadas, ya que la calidad del *test* depende del contexto y el rol que le des al modelo.

### 1. Prompt de Rol Específico

Define a la IA como un especialista para asegurar que el código generado sigue las mejores prácticas.

**Prompt Inicial:** "Actúa como un **Ingeniero de Automatización QA** experto en **TDD** y en el framework **Jest**."

### 2. Inyección de Contexto y Framework

La IA necesita saber exactamente **qué probar** y **cómo probarlo**.

| Objetivo                     | Prompt Estratégico                                                                                                                                                                                                                              |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Backend (Integración)</b> | "Usando <b>Supertest</b> , genera una prueba de integración para la siguiente ruta de Express. <b>[CÓDIGO DE RUTA]</b> . La prueba debe simular la función <code>service.createUser</code> (mockeada) y verificar el código de respuesta HTTP." |

|                          |                                                                                                                                                                                                                   |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Frontend<br>(Componente) | "Genera pruebas de interacción para el siguiente componente de React. Usa <b>React Testing Library (RTL)</b> y <code>userEvent</code> para simular que el usuario rellena el formulario y hace clic en 'Submit'." |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

### 3. Solicitud de Cobertura Específica

Utiliza la técnica *Chain of Thought* (CoT) para asegurar que la IA cubre todos los casos:

"Antes de darme el código, lista los 4 casos de prueba (éxito, error, dato faltante, dato inválido) que cubrirás en la prueba de integración. Luego, genera el código completo para cada caso."

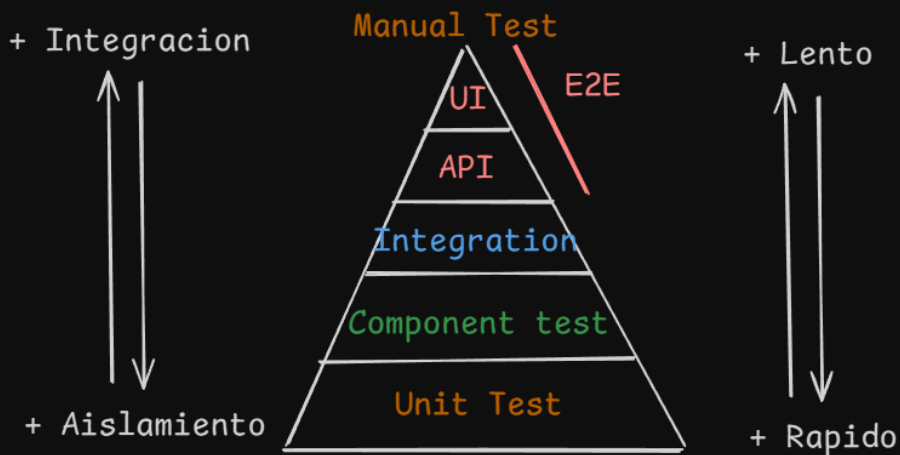
---

## V. Elementos a Probar en tu Stack

| Elemento             | Tipo de Prueba Dominante                                               | Herramientas Comunes                       |
|----------------------|------------------------------------------------------------------------|--------------------------------------------|
| Backend<br>(Express) | Integración (Rutas y Middlewares) y Unitario (Servicios).              | Jest, <b>Supertest</b> .                   |
| Frontend<br>(React)  | Componente (Renderizado y estados) y Unitario ( <i>Custom Hooks</i> ). | Jest, <b>React Testing Library (RTL)</b> . |
| Full-Stack           | E2E (Flujo de usuario completo).                                       | Cypress, Playwright.                       |

# Vibe Coding

Testing -> Probar Tests



## - Unit Test

Testea una unidad minima como una funcion

## - Component test

Testea varias unidades minimas relacionadas

## - Integration test

Testea diferentes partes del sistema que se relacionan entre si. (Interfaces)

X ej middleware -> controller

- End to end -> Simulan un flujo completo del usuario de principio a fin.

(Prueba de caja negra -> solo se preocupa del funcionamiento)

La cobertura en testing es una métrica que mide el porcentaje de código fuente de una aplicación que es ejecutado por las pruebas automatizadas.

TDD : Test-Driven Development

Es una practica donde primero se escriben los tests y luego se escribe el codigo funcional

## Pruebas en express

- Controllers (res, req)
- Servicios
- Middlewares
- Utils
- Conexiones

## SWAGGER

Test Suites: Son grupos de pruebas  
Test: Son las pruebas en si